

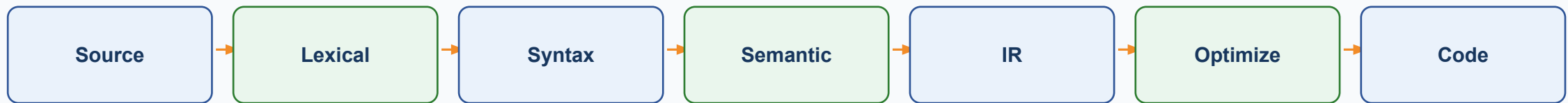
Stages of a Compiler

UG CSE lecture deck based on standard Aho–Ullman compiler organization

Compiler Design

Front → Middle → Back End

With running examples



Original teaching slides • concepts paraphrased, no copied book text

Learning objectives

What students should be able to do

Explain

- Major compiler phases
- What each phase consumes and produces
- Why errors are detected in different stages

Trace

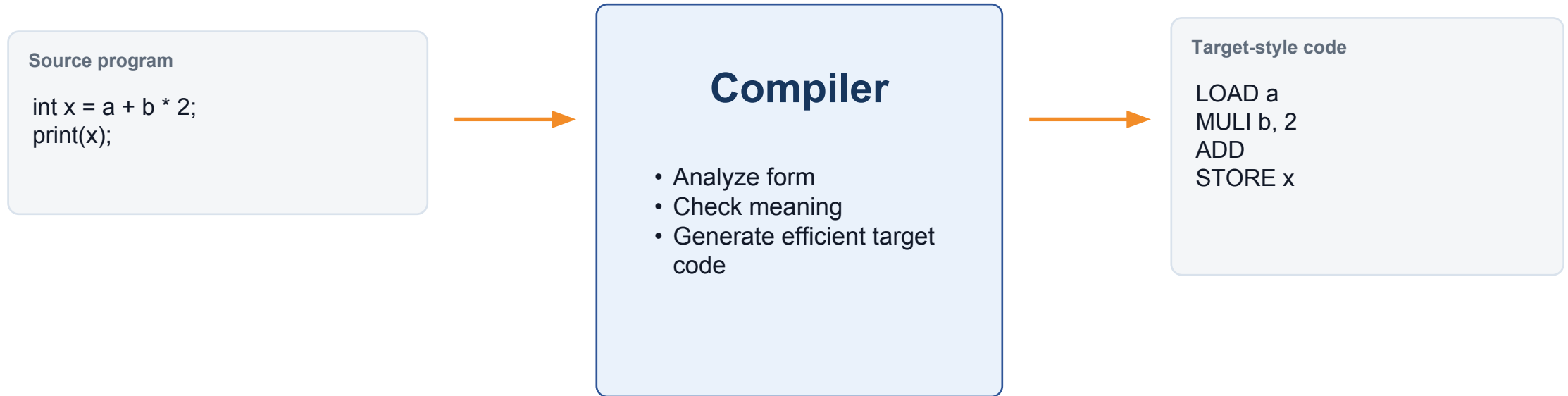
- A small program fragment from text to tokens
- Parse tree to typed IR
- IR to target-style instructions

Design

- Regular expression for tokens
- Simple grammar rules
- Basic optimizations and register choices

Compiler at a glance

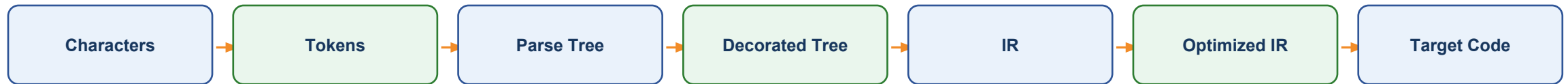
Translation with meaning preservation



Compiler correctness: the target program should behave like the source program for valid inputs.

The standard compiler pipeline

Sequence of representations



Phase	Main question	Output
Lexical	What are the words?	Tokens
Syntax	Is the sentence grammatical?	Parse tree
Semantic	Does it make sense?	Types & attributes
Optimization	Can it be improved safely?	Better IR
Code generation	How to run on machine?	Target code

Front end, middle end, back end

Why compilers are organized in parts

Front end

- Machine independent analysis
- Language syntax and meaning checks
- Builds symbol table and IR

Middle end

- Works mainly on IR
- Improves performance safely
- Keeps machine independence as far as possible

Back end

- Machine dependent choices
- Instruction selection and registers
- Produces assembly/object code

Running example for the deck

A tiny statement followed through the phases

Source statement

```
rate = base + bonus * 2;
```

Tokens

```
id(rate) = id(base) +  
id(bonus) * num(2) ;
```

Identifier

Operator

Identifier

Operator

Identifier

Operator

Number

We will use this example to see how the same source text is transformed into increasingly precise internal forms.

Stage 1: Lexical analysis

Characters → token stream

Input characters

```
rate = base + bonus * 2;
```

Token stream

```
<id, rate>  
<assign, =>  
<id, base>  
<plus, +>  
<id, bonus>  
<mul, *>  
<num, 2>
```

Lexer responsibilities

- Group characters into lexemes
- Classify lexemes as token classes
- Ignore whitespace/comments where appropriate
- Record token attributes such as identifier names and numeric values

Tokens, lexemes and patterns

Aho–Ullman style vocabulary, paraphrased

Term	Meaning	Example
Token	Category used by parser	identifier
Lexeme	Actual character sequence	bonus
Pattern	Rule describing lexemes	letter(letter digit)*

Key idea: the parser should not worry about raw characters; it receives clean, classified tokens.

Regular expressions and finite automata

How lexical rules become executable scanners



- Regular expressions describe token families compactly.
- Finite automata decide whether a character sequence belongs to a token family.
- Modern lexer generators automate much of this conversion.

Lexical errors and recovery

When characters do not form valid tokens

Problematic input

```
rate = base + @bonus;  
              ^
```

Common recovery choices

- Report unexpected character
- Skip character and continue
- Insert/delete a likely character only with care
- Keep line and column location for diagnostics

Goal: continue analysis without hiding real errors

Good diagnostics are part of compiler usability

Stage 2: Syntax analysis

Tokens → parse tree

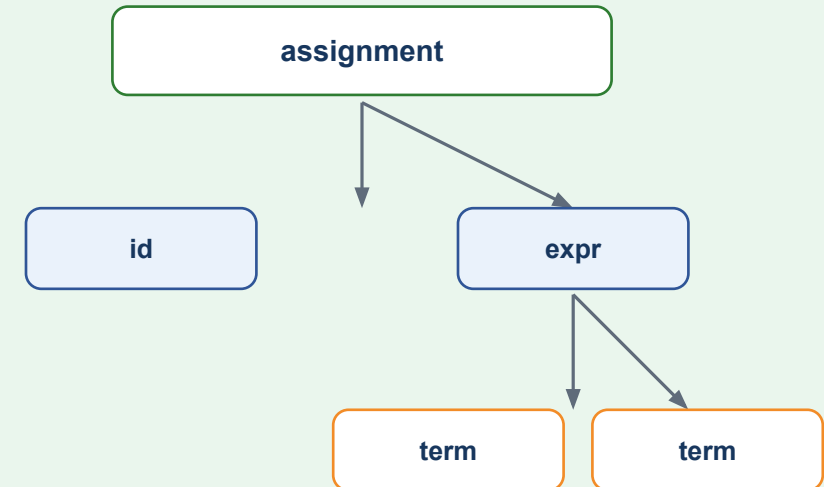
Token stream

id(rate) = id(base) + id(bonus) * num(2) ;

- Checks grammatical structure
- Imposes hierarchy on operators and phrases
- Provides a foundation for semantic checks



Parse tree idea



Context-free grammar (CFG)

Formal description of programming-language
structure

Tiny expression grammar

```
stmt → id = expr ;  
expr → expr + term | term  
term → term * factor | factor  
factor → id | num | ( expr )
```

CFG components

- Terminals: actual tokens
- Non-terminals: syntactic categories
- Productions: replacement rules
- Start symbol: entry point

A grammar separates “what forms are allowed” from “how those forms will be parsed.”

Derivation, parse tree and ambiguity

One string, possibly many structures

Derivation

Sequence of rule use

expr

⇒ expr + term

⇒ term + term

⇒ id + term

⇒ id + num

Parse tree

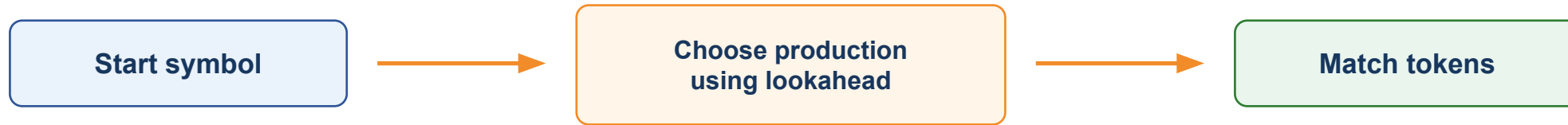
- Tree view of the same derivation
- Internal nodes: non-terminals
- Leaves: tokens/terminals

Ambiguity

- A grammar is ambiguous if one string has multiple parse trees
- Operator precedence and associativity rules usually remove ambiguity

Top-down parsing idea

Predict the input from the start symbol



- Recursive descent and LL parsing are common top-down styles.
- Parsing begins with the grammar start symbol and tries to construct the input.
- Works best when the next token guides the production choice clearly.

Mental model: expand grammar rules downward

Bottom-up parsing idea

Reduce input back to the start symbol



- LR-family parsers are bottom-up parsers used widely in compiler generators.
- They read tokens left-to-right and build the tree from leaves toward the root.
- A stack holds partial structures while actions decide shift or reduce.

Mental model: recognize completed phrases upward

Syntax error handling

Helpful compilers recover and report clearly

Example

```
rate = base + bonus * ;  
           ^ missing operand
```

Do not cascade hundreds of fake errors from one missing token.

Recovery strategies

- Panic-mode: skip until a safe token
- Phrase-level: make a local repair
- Error productions: encode common mistakes
- Global correction: theoretical but expensive

Stage 3: Semantic analysis

Parse structure → checked meaning

Questions semantic analysis answers

- Is every identifier declared?
- Are operands type-compatible?
- Is a function called with valid arguments?
- Are control statements used in legal places?

Outputs added to the tree/IR

- Type annotations
- Symbol-table links
- Implicit conversions, when legal
- Semantic diagnostics with source locations

Syntax says “well-formed”; semantics says “meaningful.”

Symbol table

Compiler memory for names and attributes

Name	Kind	Attributes
rate	variable	type:int, scope:local
base	variable	type:int, scope:local
bonus	variable	type:int, scope:local
print	function	returns:void, params:int

Used by many phases

- Lexer may insert identifiers
- Semantic analyzer checks declarations and types
- Code generator needs layout and storage details

Type checking and conversions

Prevent invalid operations early

Source fragment

```
int i;  
float f;  
f = i + 2.5;
```



Type rules

- Each expression receives a type
- Operators require compatible operands
- Safe widening conversions may be inserted
- Illegal narrowing or incompatible types produce diagnostics

Semantic analyzer protects the generated code from nonsense.

Stage 4: Intermediate representation (IR)

A bridge between language and machine



- IR should be easy to generate, easy to optimize, and easy to translate to target code.
- Three-address code, syntax trees, and SSA-like forms are common ways to represent programs internally.

Syntax-directed translation

Attach actions to grammar structure

Illustrative rule + action

```
expr → expr1 + term  
  expr.place = newTemp()  
  emit(expr.place = expr1.place + term.place)
```

Main concept

- Grammar tells the compiler where phrases are.
- Semantic actions compute attributes or emit IR.
- Attributes flow through the parse tree during translation.

Good for explaining how parsing connects to code generation.

Three-address code (TAC)

Simple statements with at most one operator

Source expression

`rate = base + bonus * 2`



Three-address code

```
t1 = bonus * 2  
t2 = base + t1  
rate = t2
```

- TAC exposes computation order and temporary values.
- It is easier to optimize than a deeply nested parse tree.
- Later phases map each statement to machine instructions.

Control flow and basic blocks

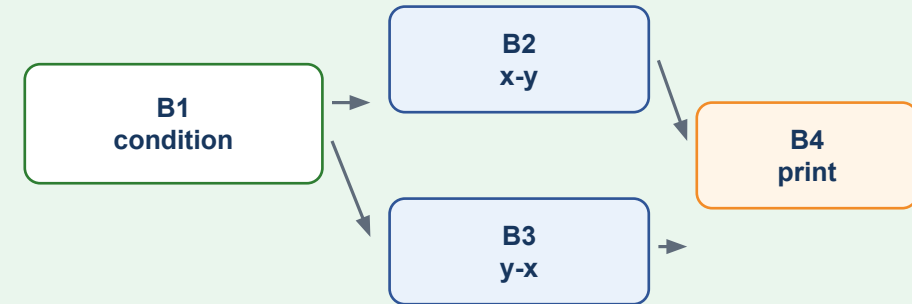
IR is organized for analysis

TAC with branches

```
if x < y goto L1  
t = x - y  
goto L2  
L1: t = y - x  
L2: print(t)
```



Control-flow graph intuition



Stage 5: Code optimization

Improve without changing program meaning

Optimization objectives

- Reduce execution time
- Reduce memory use
- Reduce code size
- Use machine resources better

Correctness rule

- Only change the form of the program, not its observable result.
- Optimizations rely on proven facts from analysis.
- Unsafe assumptions can create wrong-code bugs.

Optimization is optional for execution, essential for performance.

Local optimization examples

Small transformations students can trace

Before	Optimization	After
$x = 3 * 4$	constant folding	$x = 12$
$y = z + 0$	algebraic simplification	$y = z$
$t1 = a + b; t2 = a + b$	common subexpression	reuse t1
$u = v; w = u$	copy propagation	$w = v$

Each example is legal only when it preserves the program's meaning in its context.

Typical questions

- Which definitions can reach this point?
- Which variables are live after this statement?
- Which expressions are already available?

Why it matters

- Dead-code elimination needs liveness.
- Register allocation needs live ranges.
- Common subexpression elimination needs availability information.

Think of data-flow as “program facts at each point.”

Stage 6: Code generation

IR → target instructions

Optimized IR

```
t1 = bonus * 2  
t2 = base + t1  
rate = t2
```



Target-style sequence

```
LOAD R1, bonus  
MULI R1, 2  
ADD R1, base  
STORE rate, R1
```

- Instruction selection: choose target instructions.
- Register allocation: decide what stays in registers.
- Instruction scheduling: order operations to reduce stalls where applicable.

Register allocation

A scarce-resource problem in the back end

Why registers matter

- Registers are much faster than memory.
- Too many live values cause spilling to memory.
- Allocation must respect target-machine constraints.

Live range view

t1 live

t2 live

x live

- Values whose lifetimes overlap cannot use the same register.
- Graph-coloring and linear-scan are common allocation approaches.

Runtime storage organization

Where program data lives during execution

Memory areas

Code

Static data

Heap

Stack frames

Compiler responsibilities

- Lay out local variables and temporaries
- Generate call/return sequences
- Handle parameter passing and return values
- Respect target ABI conventions

Final output and system tools

Assembler, linker and loader complete the journey



- Compiler output is often assembly or object code, not always a directly runnable program.
- Assemblers translate assembly to machine-level object files.
- Linkers combine object files and libraries; loaders place executables in memory for execution.

Compiler design connects theory, systems, architecture and programming languages.

Reference basis: standard compiler phase organization as presented in Aho, Lam, Sethi & Ullman / Aho & Ullman compiler texts; all examples are original and paraphrased.